

JB AGENTIC MEETUP #2 · JUN 27, 2025 · SKUDAI, JB

Claude Plans, Codex Builds.

My AI-first dev workflow as a student —
what I built under pressure, what
broke,
and what I'm still figuring out.

SAMY · UTM CS

FREE TIER CLAUDE

BORROWED CODEX

FIGURING IT OUT AS I GO

WHO'S THIS GUY

Hi, I'm Sammy

BearTahan

Gamified KSSR learning app for Malaysian Standard 1 kids. Flutter + Firebase. 5-person student team. University project *and* national competition running at the same time.

Scrum Master + dev

Sprint planning, JIRA, backlog, standups — plus writing actual code. Multiple other university projects happening in parallel. No senior dev to ask.

Dr. Sim → Huiming

My supervisor connected me here. Happy to share what I figured out — and equally curious to hear how everyone else works.

No budget. Free tier Claude. Borrowed Codex from my supervisor. Had to figure out how to make AI *actually work* for a real sustained project — not just one-off demos.

forget

THE PROBLEM

AI is stateless. Your project is not.

Copy-paste the same long prompt every single session

Hit token limits constantly on free tier

AI re-suggests things your team already rejected

Spending more time re-explaining than building

what actually happened

"Claude kept re-suggesting a feature we'd killed two sprints ago. Same suggestion. Same reasoning. Fully convinced."

We were basically arguing with an AI that had **amnesia**.

That's when I realised — the problem wasn't the AI. I just wasn't giving it any memory.

AI is stateless. Fix the memory problem and everything changes.

FIX 01 give AI the memory it doesn't have

A memory system. For AI.

7 markdown files. One project. Always in context. Never re-explained.

BT-00 overview

BT-01 features

BT-02 decisions ✓

BT-03 backlog

BT-04 sprints

BT-05 tech notes

BT-06 open items

Claude Projects

Every chat auto-loads context — no re-pasting ever.

Start a fresh chat when context gets heavy.

Files persist.

Hit usage limit? Switch account, same folder, continue.

Ping a prompt at the start of each new 5-hour window.

Codebase Folder

Same files placed inside the repo.

The coding agent reads them directly from disk.

No copy-pasting. No re-explaining.

Both tools share the exact same project memory.

Not docs for humans. Memory for AI. **Quality of context = quality of every output.**

FIX 02 one role each · play to their actual strengths

Two agents. Neither does everything.

Claude

city planner · sees the whole map

Plans features & architecture

Spots edge cases before they happen

Challenges assumptions & decisions

Writes handoff notes for Codex

Big picture thinking

YOU
PM

in the
middle

Codex

mechanic · one thing at a time

Writes the actual Flutter code

Lives inside the terminal + codebase

Loops until the build passes

Reads the same context files

Executes the plan precisely

Feels like **senior engineer** ↔ **you as PM** ↔ **junior engineer**

UNEXPECTED BONUS

They catch each other's *blind spots.*

Claude notices

Logical gaps and architectural problems Codex would miss — because Codex is too close to the code to see the bigger picture.

Codex notices

Things that look fine on paper but break in the real codebase. Theory vs implementation reality.

"Claude would warn me the architecture might break later. Codex would just go and implement it."

Seeing both reports gives you more than either one alone. One challenges your thinking. One executes your thinking. You sit in the middle and decide which is right.

AGENTS.MD the rulebook · markdown file living in the codebase

At some point I needed to write down how the two tools should work together — not just what context to load, but how they should *talk* to each other.

AGENTS.md · Claude Collaboration Protocol

```
// When Codex needs Claude, produce a prompt with:  
current goal & acceptance criteria  
existing implementation & constraints  
decisions made · alternatives rejected  
unresolved questions & risks
```

*"Treat Codex's conclusions as proposals,
not demands. Challenge assumptions
constructively. The goal is to widen
our combined perspective and cover
each other's misses — not establish authority."*

```
// peers. human coordinates. no hierarchy.
```

Developer Mode

Codex focuses on implementation, debugging, Firebase, tests, Git. Explores before asking.

PM / Scrum Master Mode

Sprint goals, backlog quality, story clarity, dependencies, demo readiness. No code unless asked.

DEVELOPER MODE

PM / SCRUM MASTER MODE

AGENT SWITCHES BEHAVIOUR BASED ON CONTEXT

Automation is safe when **all decisions are already made.**



Why not full automation?

Errors compound. One hallucination becomes input for the next step. By the time you notice, you're 3 steps deep in the wrong direction. Unwinding that is slower than checking at step one. Intent also drifts — quietly, confidently.

The principle

The moment there's a judgment call — product, architecture, what makes sense — I stay in the loop. This boundary doesn't move regardless of tools. If I switch to Claude Code, context auto-updates. The human part stays. As tools improve, manual parts shrink. Philosophy stays.

THE PART NOBODY TALKS ABOUT

AI didn't just help me code. It helped me manage the team.

What AI helped with as SM

Sprint plans + JIRA story setup

User stories + acceptance criteria

Dependency coordination between devs

Sprint briefing messages to team

Branch integration across 5 developers

the team moment

"When a teammate asked how a feature worked, I told them to load the MD files into their own AI and ask there."

One teammate set up NotebookLM with all the context modules → became the team's shared knowledge base. Anyone could query the project without asking me.

I didn't want to be the bottleneck for every question.



DIGITEX 2026

Gold Medal.

Top 20 of 131
competing teams.

National University Innovation Competition · UTM
Faculty of Computing

I'm not saying the workflow is the
reason.

But without it, we wouldn't have
shipped something complete
enough to compete.

Five students. One direction. Four sprints.

**"I didn't set out to build
a system.
I just didn't want to
repeat myself every
day."**

found out later it has industry names — AGENTS.md · context engineering · spec-driven
development

didn't know any of that going in · still figuring it out · will look different in 3 months

GIVE AI MEMORY

ONE ROLE PER TOOL

STAY IN THE LOOP

START WITH MARKDOWN

ask

let's talk.

- WHY NOT CLAUDE CODE?
 - WHY NOT FULL AUTOMATION?
 - HOW DO YOU KEEP FILES UPDATED?
 - WHAT BROKE THE MOST?
 - HOW DOES YOUR TEAM USE IT?
 - ONE AGENT VS TWO?
 - WHAT WOULD YOU DO DIFFERENTLY?
- Samy · UTM CS · JB Agentic Meetup #2